14 10月

# 每日晨间简报

MORNING LINE · DAILY BRIEF

星期三 · 2026年10月14日

本站 → 终点:填写记录

新闻 GitHub 打卡 项目 CSES 重写 Rust 精读 原著 记录

> **锲而舍之,朽木不折;锲而不舍,金石可镂。**
>
> —— 荀子《劝学》(刻几下就停,烂木头也刻不断;一直刻下去,金属和石头也能雕出花纹。)

今日一首歌

**Clocks — Coldplay**

英国乐队 2002 年的作品,一段反复的钢琴分解和弦贯穿全曲,节奏稳定。适合上午写代码前听一遍,或者做题时当背景。

**本站** 今天做 chat-cli 第 4 天:流式输出,练习 CSES 1617 Bit Strings。

## 一 今日三条新闻

### 1 Google 发布 Gemini 4 Argon:一次最多处理 100 万个词元

Google 官方博客 10月13日宣布,开发者和企业可以通过 Gemini API 和 AI Studio 使用 Gemini 4 Argon,Pro 和 Ultra 订阅用户在 Gemini 应用里使用。官方称它通过 API 有 100 万词元的输入输出总上限;优惠价是每百万输入词元 2 美元、输出 10 美元,11月30日后涨到 4 美元和 20 美元。(AIHOT)

**为什么值得看:**这个“上限”就是上下文窗口,今天英语精读讲它;你的 chat-cli 每轮重发全部历史,用的正是这个窗口。

### 2 Anthropic 自研版本控制系统 Scion,五个月换掉 git

Anthropic 工程博客 10月13日称,公司每天合并的代码改动从 2026年5月约 2,500 次涨到 9 月超过 30,000 次,版本控制成了最大的开发瓶颈。一个 7 人团队用五个月做出 Scion 和代码评审工具 Antique,把整个单一代码仓库从 git 迁了过去;文章称它的 CPU 和内存占用比原来的 git 镜像设施小 200 倍。(AIHOT)

**为什么值得看:**AI 写的代码多了,连 git 这种基础工具都要重新设计;每天 git commit 时可以想想它背后要处理多大的规模。

### 3 美股 13 日基本持平:银行财报有好有坏,芯片股走弱

TheStreet 10月13日收盘报道:道指跌 0.35% 收于 51,670.10,标普 500 跌 0.02% 收于 7,805.45,纳指跌 0.15% 收于 27,204.38。摩根大通每股收益 5.57 美元,低于预期的 5.90 美元;高盛 17.01 美元,高于预期的 13.02 美元。报道称,中国长鑫存储(CXMT)推进自研内存芯片设计的消息让美光、SK 海力士和三星早盘下跌。

**为什么值得看:**AI 模型离不开内存芯片,芯片行业的竞争最后会落到模型和 API 的价格上。

所属方向:1 科技与 AI · 2 编程与开发 · 3 财经与市场

## GITHUB 今日项目

### atuinsh / atuin

用 Rust 写的 shell 历史记录工具:把你在终端里敲过的命令存进 SQLite 数据库,按 Ctrl-R 打开全屏搜索。

Rust | 约 3.2 万星 | 创建于 2020年10月4日 | MIT 许可证 | 成熟项目,这一周仍有提交

**有什么用**

- 每条命令都记下退出码、所在目录、主机和耗时,可以只搜当前目录或当前会话里的命令。
- 可选:在几台机器之间同步历史,README 说是端到端加密,也可以自建服务器或干脆不同步。

**为什么火**

- 2020 年开始,积累了约 3.2 万星,到这周还在更新,是 Rust 写的终端工具里的常青项目。
- 支持 zsh、bash、fish、nushell、xonsh、PowerShell 六种 shell,原来的历史文件保留不动。

**怎么用**

```
curl --proto '=https' --tlsv1.2 -LsSf https://setup.atuin.sh | sh
atuin import auto   # 导入已有的历史;重启 shell 后按 Ctrl-R 搜索
```

**要注意的**

README 快速开始里的 `atuin register` 会注册官方同步服务,不想上传历史就跳过它和 `atuin sync`;bash 下依赖 bash-preexec,有些限制。以上来自 README 和 GitHub 统计,未实际安装。

**和你的关系**

你每天反复敲 `cargo run`、`curl -v`、`codecrafters test`,按目录搜历史能少打很多字。

## 今日任务

标“必做”的是项目(chat-cli 第 4 天)、今日练习(今天是 CSES)、隔天重写和填写记录;其余选做,做到哪算哪。

**打卡 · 复利** 从 2026年10月10日 起算 · 截至昨天

**×1.198** 总累计

本月(10月):**×1.198**

本月 1.1 有 **3** 天,0.9 有 **1** 天

往月:—

已持续 **4** 天(10月10日起)

连续 1.1:第 **3** 天 · 状态:**未断**

**2026年10月**

| 一 | 二 | 三 | 四 | 五 | 六 | 日 |
|---|---|---|---|---|---|---|
| | | | 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 | 9 | 10 0.9 | 11 1.1 |
| 12 1.1 | 13 1.1 | 14 ? | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 | |

黑底 = 1.1 · 虚线框 = 0.9 · 点线框 = 今天 · 灰底 = 起算之前 · 细边空白 = 以后

☐ 今天:项目、CSES 或 CodeCrafters 有进展(完成或部分完成),或做了隔天重写,就是 1.1——总累计变成 **1.318**;一项都没动是 0.9——变成 **1.078**。

**回你的备注**

13 日是到现在记录最满的一天:项目、1092、ia4、重写 1072、读书、力量训练都有。

**ia4** 是你第一次独立过的 CodeCrafters 关卡,一步一行注释,写得清楚。一个小地方:`write_all` 成功时返回的是 `()`,所以 `println!("{response:?}")` 只会打印 `()`,这一行可以删。

**1092** 超时那次值得记住:在循环里每次重新求和,等于 n 次求和套在 n 次循环里;放到外面、每轮只加 k,就只走一遍。另外 `vec1_sum + k <= medium` 已经包含前半个条件,只留它就够。

**chat-cli** 这次选的是“问了 AI”,那 7 条卡点读起来更像 AI 替你写的复盘。最关键的是第 3 条(借用里的东西不能拿走)和第 5 条(回复要放回历史)。今天 Rust 讲“引用与借用”,专门解决第 3 条;项目卡片开头加了两条方向提示。建议卡住时先自己试 20 分钟,把报错原文抄进记录,再去问。

“悲观没有意义”——同意。比起想状态,做完一件记一件更管用,你这几天的记录就是这样。

**从这里开始** **chat-cli 第 4 天 第 1 条:**请求体里把 `stream` 改成 `true`,先只把收到的原始字节打印出来。

### 项目线 ☐ chat-cli 项目 · 第 4 天:流式输出 必做

**先看这里** · 昨天 7 条卡点里的第 2、3 条今天还会碰到:拼完整回复用 `reply.push_str(text)`(追加一段字符串),不要再 collect 成新的 Vec;`choices.first()` 给的是借用,只读文字时写 `if let Some(text) = &choice.delta.content`,不需要把它拿走。
**方向提示 1:**先老老实实做完第 1–3 条,亲眼看到“一块不等于一行”,再写缓冲区。**方向提示 2:**缓冲区里找一行,就是找 `b'\n'` 的位置,把它之前的字节切出来、剩下的留着;切出来的那一行才转成字符串。

**目标:**回复不再等全部生成完才显示,而是边生成边打印。这是一周里最难的一天。

**加依赖:**

```
cargo add reqwest --features json,stream
cargo add futures-util
```

### 先理解格式

请求体里把 `stream` 改为 `true` 后,服务器返回的不再是一个 JSON,而是一行一行的文本,这种格式叫 SSE(Server-Sent Events):

```
data: {"choices":[{"delta":{"role":"assistant","content":""}}]}

data: {"choices":[{"delta":{"content":"你"}}]}

data: {"choices":[{"delta":{"content":"好"}}]}

data: [DONE]
```

规则只有三条:每条数据以 `data: ` 开头;数据之间隔一个空行;遇到 `data: [DONE]` 表示结束。每条数据里的新增文字在 `choices[0].delta.content`。

### 分两步做 · 第一步,先看原始数据

**第1条** ☐ 请求体加 `stream: true`

**第2条** ☐ 用 `resp.bytes_stream()` 拿到字节流,循环读取,把每块原样打印出来

**第3条** ☐ 观察:一块里可能有好几行,也可能一行被切成两块

### 第二步,解析并逐字打印

**第4条** ☐ 准备一个 `Vec<u8>` 缓冲区,把每块字节追加进去

**第5条** ☐ 每次追加后,把缓冲区里所有完整的行(以 `\n` 结尾)取出来,不完整的尾巴留在缓冲区

**第6条** ☐ 对每一行:空行跳过;去掉 `data: ` 前缀;是 `[DONE]` 就结束;否则解析成 JSON

**第7条** ☐ 定义 `StreamChunk`、`StreamChoice`、`Delta` 三个结构体

**第8条** ☐ 打印 `delta.content` 并立刻 `flush`

**第9条** ☐ 同时把所有片段拼成完整回复,结束后追加进历史

### 提示

- `use futures_util::StreamExt;` 之后才能写 `while let Some(chunk) = stream.next().await`。
- `Delta` 里的 `content` 用 `Option<String>`,有的数据块里没有它。
- 取第一个 choice 用 `choices.first()`,不要用 `choices[0]`,有的数据块 `choices` 可能是空的。
- 去前缀用 `line.strip_prefix("data: ")`。

### 常见坑

- **一个网络数据块不等于一行。**这是今天最容易错的地方,所以才需要缓冲区。
- **中文字符可能被切开。**一个汉字占 3 个字节,可能前 2 个在这一块、后 1 个在下一块。所以缓冲区用 `Vec<u8>`,取出完整的一行之后再转成字符串。
- 忘记 `flush`,文字会攒在一起最后才显示,看起来像没有流式。
- 回复结束后记得补一个换行。

**完成标准** 让模型写一段 200 字的文字,能看到它逐步出现,中文没有乱码;接着追问一句,模型仍记得刚才写的内容。

照你的 7 天指南 第 4 天写。做完后在记录页把项目标为完成,简报才会进到第 5 天;卡住时把做到第几条和报错填进项目那一栏。

练习线 ☐ **CSES 1617 Bit Strings** 必做

### 思路

这是一道计数题。长度为 n 的串,每一位都有两种选法,而且各位互不影响,所以按乘法原理把每一位的选法数乘起来。真正要处理的是:n 最大 100 万,结果大得装不进任何整数类型,题目只要它除以 $10^9+7$ 的余数——想想“每一步都只保留余数”为什么不影响最后的余数。

1. **手算小例子,找规律。**
   写出 n = 1、2、3 时所有的串,数一数个数。
   > **怎么确认:**n=1 是 2 个,n=2 是 4 个,n=3 是 8 个,和题目样例一致;能用一句话说出“为什么每多一位就翻倍”。

2. **读入 n。**
   输入只有一个整数,读进来先原样打印。
   > **怎么确认:**输入 3 打印 3。

3. **边乘边取余。**
   从 1 开始,循环 n 次,每次乘 2 后马上对 $10^9+7$ 取余。
   > **怎么确认:**n=3 输出 8,n=10 输出 1024。

4. **检查会不会溢出。**
   想一下:取余后的值最大是多少?再乘 2 会不会超出你选的类型?
   > **怎么确认:**n=30 输出 `73741817` ($2^{30}$ = 1073741824,减去一次 $10^9+7$);n=31 时 $2^{31}$ 已经超过 i32 上限,用 u64 不会出事。

5. **测最大数据,再提交。**
   输入 1000000,看运行时间。
   > **怎么确认:**几乎瞬间输出 `235042059`,然后提交。

### 相关知识

- **乘法原理:**每一步的选法互不影响时,总数 = 各步选法数相乘。CSES 后面很多计数题都从这里开始。
- **取余可以提前做:**(a · b) mod m = ((a mod m) · (b mod m)) mod m,所以每乘一次就取余,数永远不会变大。明天英语原著读的 1.3 节正好讲这条。
- **Rust 写法要点:**模数写成常量 `const M: u64 = 1_000_000_007;`,下划线只是为了好读;整数里不要写 `1e9 as u64 + 7` 这种浮点转换。
- **不必用快速幂:**n 只到 $10^6$,循环 n 次就够快;快速幂以后遇到 n 到 $10^{18}$ 的题再学。

---

明天:CodeCrafters HTTP 服务器 ih0 Extract URL path(CSES 停在你做到的那一题)。

重写线 ☐ **隔天重写 · CSES 1092 Two Sets + CodeCrafters ia4** 必做

**今天要重写的:**13 日做完的 CSES 1092 和 HTTP 服务器 ia4 Respond with 200。

**做法:**合上昨天的代码,新建文件从空白开始;先用中文注释写出每一步,再一行行翻成 Rust;1092 用样例验证,ia4 用 `curl -v http://localhost:4221` 看到 `HTTP/1.1 200 OK`;写不出来的地方记进记录页“重写时哪里又写不出来了”。

**对照清单 · 1092**

- ☐ 先判断 1 到 n 的总和是不是奇数,奇数直接输出 NO
- ☐ 从 n 往下贪心放进第一组,直到和正好是一半;和只算一次,每轮加 k
- ☐ 输出格式:YES、第一组个数、第一组元素、第二组个数、第二组元素

**对照清单 · ia4**

- ☐ bind 到 127.0.0.1:4221,incoming 循环加 match 的 Ok / Err
- ☐ `Ok(mut stream)` 和 `use std::io::Write`
- ☐ 用 write_all 写出状态行和结尾的空行(两个 \r\n)

重写完在记录页勾上“重写了前一天的 CSES 题”和“CodeCrafters 关卡”。

Rust 线 ☐ **Rust 知识点 · 第 4 讲:引用与借用** 选做

**一句话:**想让一个函数“看一看”或“改一改”某个值,又不想把这个值整个交出去,就借给它——`&x` 是只读地借,`&mut x` 是可以改地借。借完东西还在你手里。

## 第一部分 · 书里讲的

- **引用就是“借”。**第 1 讲说过,把 String 传给函数会把所有权移走,之后原变量不能再用。写成 `&s` 传进去,函数拿到的只是一个指向它的引用,用完 s 还是你的。
- **默认只读。**`&T` 只能读不能改。想改,变量本身要 `let mut`,传的时候写 `&mut s`,函数参数写 `&mut String`。
- **借用规则:**同一时间,要么有任意多个只读引用,要么只有一个可变引用,两者不能同时存在。这条规则在编译时检查。
- **借用到“最后一次使用”为止。**引用不是活到大括号结束,而是活到它最后一次被用的那一行;之后就可以再借。
- **不会有悬空引用。**引用不能比它指向的值活得更久;函数里新建的值不能把引用返回出去,编译器会拦下来。

**例 1(能编译):**只读借和可变借。输出 `2 hi!`。

```
fn len_of(s: &String) -> usize {
    s.len()
}
fn add_bang(s: &mut String) {
    s.push_str("!");
}
fn main() {
    let mut s = String::from("hi");
    let n = len_of(&s);
    add_bang(&mut s);
    println!("{n} {s}");
}
```

**例 2(这段编不过):**就是你 chat-cli 第 3 条卡点。`first()` 给的是借来的 `&Choice`,不能把里面的 String 拿走。报 `E0507: cannot move out of `first.text` which is behind a shared reference`。

```
struct Choice {
    text: String,
}
fn main() {
    let v = vec![Choice { text: "hi".to_string() }];
    let first = v.first().unwrap();
    let t: String = first.text;
    println!("{t}");
}
```

**例 3(能编译):**两种改法。只想看,就借着看(`&c.text`);确实要拿走,就让 Vec 交出所有权(`into_iter()`),之后 v 不能再用。输出“借来看: hi”和“拿走了: hi”。

```
struct Choice {
    text: String,
}
fn main() {
    let v = vec![Choice { text: "hi".to_string() }];
    if let Some(c) = v.first() {
        let t: &str = &c.text;
        println!("借来看: {t}");
    }
    let first = v.into_iter().next().unwrap();
    let owned: String = first.text;
    println!("拿走了: {owned}");
}
```

**例 4(这段编不过):**拿着只读引用时又去改 Vec。报 `E0502: cannot borrow `v` as mutable because it is also borrowed as immutable`。

```
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    v.push(4);
    println!("{first}");
}
```

## 第二部分 · 书里没细讲,但你应该知道的

- **例 4 为什么必须拦。** `push` 可能让 Vec 搬到一块更大的内存,旧地址作废,`first` 就指向了被释放的内存。C++ 里这是真实存在的 bug(迭代器失效),程序可能照常跑、偶尔崩;Rust 在编译时直接拒绝。Python、Java 靠垃圾回收避开了这一类问题,代价是运行时开销。
- **内存里是什么。**引用就是一个地址(8 个字节),不复制数据。`&str` 和 `&[T]` 多带一个长度,叫“胖指针”。所以把 `&Vec<T>` 改成 `&[T]` 当参数,既更通用,也一样便宜。
- **Option 里套引用。** `v.first()`、`line.strip_prefix(...)`、`it.next()`(对 `iter()`)给的都是 `Option<&…>`:里面是借来的。对 `Option<String>` 想借着看,写 `if let Some(t) = &opt` 或 `opt.as_deref()`(得到 `Option<&str>`)。
- **`.clone()` 是退路,不是答案。**编译器报借用错误时,到处 clone 能编过,但会复制大量数据,也把“到底谁拥有它”的问题藏起来了。先问自己:我是只想看,还是真的要拿走?
- **两种场景。**刷题时把大数组以 `&[u64]` 传给函数,避免复制;写服务器时 `write_all` 要的是 `&mut stream`,所以 ia4 要写 `Ok(mut stream)`;`for x in &v` 是借着遍历,`for x in v` 会把 v 用掉。

### 常见错误

- `E0382: borrow of moved value`:值已经传给函数(被移走)又再用 → 参数改成引用,调用时加 `&`。
- `E0596: cannot borrow as mutable`:变量没写 `mut` 却想 `&mut` 借 → 在 `let` 后面加 `mut`。

### 小练习

下面这段报 E0382。只改函数签名和调用处两个地方,让它能编译并输出 `6 [1, 2, 3]`。

```
fn total(v: Vec<u64>) -> u64 {
    v.iter().sum()
}
fn main() {
    let v = vec![1, 2, 3];
    let s = total(v);
    println!("{s} {:?}", v);
}
```

**和今天任务的联系:**chat-cli 第 4 天全是借用:`choices.first()` 返回 `Option<&StreamChoice>`,`&delta.content` 借着读文字再 `push_str` 进回复;`strip_prefix` 返回的 `&str` 借的是那一行,要在这一行还在的时候用完。CSES 1617 用不太到这一讲。

练习答案:签名改成 `fn total(v: &[u64]) -> u64`,调用改成 `total(&v)`。 · 对应书里:4.2 引用与借用。

## Essay

# How Much Can a Model Read at Once?

¶1 Every time you send a message to a chat model, the model reads text and writes text, and both are counted in tokens. The **input tokens** are everything you send: the system message, the whole conversation history, and your new question. The **output tokens** are the reply. The **context window** is the largest number of tokens the model can handle in one request.

¶2 On October 13, Google released Gemini 4 Argon. Google says the model has a one-million-token **limit** for input and output together through the Gemini API. That is a lot of text: roughly several long books, or a large part of a code base. A big window lets a model look at many files at the same time, instead of guessing what is in the files it cannot see.

¶3 But a big window is not free. Google's **introductory price** is 2 dollars per million input tokens and 10 dollars per million output tokens until November 30, 2026. After that, both prices double. If you fill the window again and again, the **bill** grows quickly. That is why Google also sells **cached** input tokens at 95 percent off: when the start of a request is the same as before, the model does not have to process it again at the full price.

¶4 You have already seen this in your own chat-cli. The model has no memory, so your program sends the whole history every turn, and prompt_tokens gets bigger each time. When the history becomes longer than the window, the program has to drop or summarize old messages. This is called **truncating** the history.

¶5 So "How big is the context window?" is a useful question. "How much of it do I really need to send?" is just as useful. A good program sends what the model needs, not everything it has.

### 8 个关键词

**input tokens** 输入词元

**output tokens** 输出词元

**context window** 上下文窗口

**limit** 上限

**introductory price** 优惠(推广)价

**bill** 账单

**cached** 缓存过的

**truncate** 截断

新闻原文:Gemini 4 Argon: Built to solve your hardest problems(Google 官方博客);AIHOT 条目。

用中文写 3 句话总结:

---

---

---

### 中文对照(先读完英文再看)· 模型一次能读多少?

¶1 每次你给聊天模型发消息,模型都会读一段文字、写一段文字,两者都按词元计数。**输入词元**是你发过去的全部内容:system 消息、整段对话历史,加上你的新问题。**输出词元**是模型的回复。**上下文窗口**是模型在一次请求里能处理的最大词元数。

¶2 10 月 13 日,Google 发布了 Gemini 4 Argon。Google 说,通过 Gemini API 使用时,这个模型的输入和输出合计**上限**是一百万词元。这是很多文字:大约相当于好几本厚书,或者一个代码库的一大部分。窗口大,模型就能同时看很多文件,而不用去猜那些它看不到的文件里写了什么。

¶3 但大窗口不是免费的。到 2026 年 11 月 30 日为止,Google 的**优惠价**是每百万输入词元 2 美元、每百万输出词元 10 美元;之后两个价格都翻倍。如果你一次又一次把窗口塞满,**账单**会涨得很快。所以 Google 对**缓存过的**输入词元只收原价的 5%:请求开头和上次一样时,模型不用再按全价处理一遍。

¶4 你在自己的 chat-cli 里已经见过这件事。模型没有记忆,所以你的程序每轮都把全部历史发过去,prompt_tokens 一次比一次大。历史比窗口还长时,程序就得丢掉或总结旧消息,这叫**截断**历史。

¶5 所以,“上下文窗口有多大?”是个有用的问题;“我到底需要发多少?”同样有用。好程序发的是模型需要的东西,而不是手头的全部。

先读完英文,再看关键词,最后对照中文。

今天读:Chapter 1 Introduction · 1.2 Input and output · 书页 4–6(PDF 第 14–16 页)

**Before you read**

**Q1** How does a contest program usually get its input and give its output?

**Q2** Can reading and writing data make a program slow?

## 1.2 Input and output

¶1 In most contests, standard streams are used for reading input and writing output. In C++, the standard streams are cin for input and cout for output. In addition, the C functions scanf and printf can be used.

¶2 The input for the program usually consists of numbers and strings that are separated with spaces and newlines. They can be read from the cin stream as follows:

```
int a, b;
string x;
cin >> a >> b >> x;
```

¶3 This kind of code always works, assuming that there is at least one space or newline between each element in the input. For example, the above code can read both of the following inputs:

```
123 456 monkey
```

```
123    456
monkey
```

¶4 The cout stream is used for output as follows:

```
int a = 123, b = 456;
string x = "monkey";
cout << a << " " << b << " " << x << "\n";
```

¶5 Input and output is sometimes a bottleneck in the program. The following lines at the beginning of the code make input and output more efficient:

```
ios::sync_with_stdio(0);
cin.tie(0);
```

¶6 Note that the newline "\n" works faster than endl, because endl always causes a flush operation.

¶7 The C functions scanf and printf are an alternative to the C++ standard streams. They are usually a bit faster, but they are also more difficult to use. The following code reads two integers from the input:

```
int a, b;
scanf("%d %d", &a, &b);
```

¶8 The following code prints two integers:

```
int a = 123, b = 456;
printf("%d %d\n", a, b);
```

¶9 Sometimes the program should read a whole line from the input, possibly containing spaces. This can be accomplished by using the getline function:

```
string s;
getline(cin, s);
```

¶10 If the amount of data is unknown, the following loop is useful:

```
while (cin >> x) {
    // code
}
```

¶11 This loop reads elements from the input one after another, until there is no more data available in the input.

¶12 In some contest systems, files are used for input and output. An easy solution for this is to write the code as usual using standard streams, but add the following lines to the beginning of the code:

```
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
```

¶13 After this, the program reads the input from the file ”input.txt” and writes the output to the file ”output.txt”.

Antti Laaksonen, *Competitive Programmer's Handbook*, 第 1 章 1.2 节,CC BY-NC-SA 4.0。

---

### 8 个关键词

**standard stream** 标准流

**separated** 被(空格等)隔开的

**assuming that** 假定……

**bottleneck** 瓶颈

**flush** 刷新(把缓冲内容真正写出去)

**alternative** 替代办法

**accomplish** 完成、做到

**available** 可用的、还有的

### After you read

**Q1** Why can the same code read both "123 456 monkey" on one line and the same data on two lines?

**Q2** Why is "\n" faster than endl?

**Q3** Which loop is useful when you do not know how much data there is?

Write one sentence in English to sum up this section:

---

### 中文对照(先读完英文再看)

¶1 在大多数比赛中,读输入和写输出用的是标准流。在 C++ 里,标准流是用于输入的 cin 和用于输出的 cout。此外,也可以用 C 语言的函数 scanf 和 printf。

¶2 程序的输入通常由数字和字符串组成,它们之间用空格和换行隔开。可以像下面这样从 cin 流里读入:〔代码见原文〕

¶3 只要输入里每两个元素之间至少有一个空格或换行,这种代码就总能正常工作。例如,上面的代码既能读下面第一种输入,也能读第二种:〔代码见原文〕

¶4 cout 流的输出用法如下:〔代码见原文〕

¶5 输入输出有时是程序的瓶颈。在代码开头加上下面两行,可以让输入输出更高效:〔代码见原文〕

¶6 注意,换行符 "\n" 比 endl 快,因为 endl 每次都会触发一次刷新操作。

¶7 C 语言的函数 scanf 和 printf 是 C++ 标准流的替代办法。它们通常稍快一些,但用起来也更麻烦。下面的代码从输入中读两个整数:〔代码见原文〕

¶8 下面的代码打印两个整数:〔代码见原文〕

¶9 有时程序需要从输入里读一整行,这一行里可能有空格。这可以用 getline 函数做到:〔代码见原文〕

¶10 如果数据量未知,下面这个循环很有用:〔代码见原文〕

¶11 这个循环一个接一个地从输入中读取元素,直到输入里没有更多数据为止。

¶12 有些比赛系统用文件来输入输出。一个简单的办法是照常用标准流写代码,但在代码开头加上下面两行:〔代码见原文〕

¶13 这样以后,程序就从文件“input.txt”读输入,把输出写到文件“output.txt”里。

译文据 CC BY-NC-SA 4.0 原文翻译,仅供个人学习。

### 这一节的代码

书里的 C++ 代码做了三件事:按空白切开读数字和字符串、用 "\n" 而不是 endl 输出、关掉同步让输入输出变快。Rust 里对应的写法是:一次把全部输入读成 String,用 `split_whitespace()` 切开(空格和换行都算分隔,所以两种输入都能读);输出套一层 `BufWriter`,攒够了再一起写。Rust 的标准输出按行缓冲,大量 `println!` 会一行行写出去,就像 endl 一样慢。`while (cin >> x)` 对应 `for tok in s.split_whitespace()`。

```rust
use std::io::{self, Read, Write};

fn main() {
    let mut s = String::new();
    io::stdin().read_to_string(&mut s).unwrap();
    let mut it = s.split_whitespace();
    let a: i64 = it.next().unwrap().parse().unwrap();
    let b: i64 = it.next().unwrap().parse().unwrap();
    let x = it.next().unwrap();
    let out = io::stdout();
    let mut w = io::BufWriter::new(out.lock());
    writeln!(w, "{} {} {}", a, b, x).unwrap();
}
```

**能用在哪类 CSES 题上:**所有题都要读输入;输入或输出很大的题最用得上,比如你做过的 1092 要打印两组共 n 个数,用一个 String 拼好或者 BufWriter 一次写出,比逐个 `println!` 快得多。

**路线:**今天是 118 段里的第 2 段。上一段:1.1 Programming languages,讲比赛常用的语言和 C++ 模板。下一段:1.3 Working with numbers,讲整数类型和取余运算——正好是今天 1617 用到的东西。

Reference answers: ① Because cin only needs at least one space or newline between the elements. ② Because endl always causes a flush operation. ③ The loop while (cin >> x), which reads until there is no more data. · Reference summary: Contest programs read input and write output with standard streams, and a few small changes can make input and output faster.

- [ ] **读书:《1984》第一部 第 5 章** —— 接着你读完的第 4 章。这一章大多发生在午饭时的食堂里:留意那位编词典的同事为什么觉得词越少越好,以及主角在心里怎么评价身边这几个人。 选做
- [ ] **运动:游泳** —— 先在水里慢走或慢游热身;自由泳或蛙泳 8–10 个 25 米,每个之间歇 30–60 秒;最后放松游 100 米,上岸拉伸肩和腿。 选做

终点站 [ ] **填写今晚的记录** 必做

做了一点也写上做到第几步(项目填在记录页最上面的项目一栏,写“第 4 天第几条”);照实选“怎么完成的”,它只用来调提示多少;重写了就勾上;CodeCrafters 关卡填编号(例如 ih0);选做的做了就勾。备注里想说什么都可以写,明天的简报开头会回你;明天从这里写的位置接着排,打卡系数也按这里算。